

The Integer Programming Solution to Facility Location Problems*

Keith Head¹, Thierry Mayer², Marc Melitz³, and Chenying Yang⁴

¹University of British Columbia and CEPR

²Sciences Po, CEPII and CEPR

³Harvard University, NBER, and CEPR

⁴Singapore Management University

May 26, 2026

Download latest draft

Abstract

An important type of problem in the study of multinational corporations and global sourcing occurs when a firm must pay a fixed cost to open a production site in order to use it to provide inputs or serve consumers. The key trade-off is between producing close to demand versus avoiding fixed costs. The operations research literature terms this a *facility location problem* (FLP). The standard method for solving such problems is *mixed integer linear programming* (MILP). Here we show the usefulness of this approach, especially because it *a*) does not rely on parametric restrictions, *b*) can be extended to handle more than one stage of production and other more complex problems than the classic FLP.

*We thank Rowan Shi, Felix Tintelnot, Pablo Fajgelbaum, and Jin Liu for helpful discussions of the methodology described here. Keith Head is indebted to UBC colleagues in the Operations and Logistics division (Mahesh Nagaranjan, Joseph Paat, Julia Yan) for answering his many questions.

1 Introduction

This paper is organized as follows. Section 2 formalizes the facility location problems that are our focus here. Section 3 explains how we can be sure that the methods we use give globally optimal solutions. Section 4 examines performance in simulations. We compare MILP performance to squeezing methods in a problem that both can handle. Then we investigate performance of multi-stage location problems with fixed costs, a problem for which MILP is often the only fast way to proceed. Section 5 digs into the operations research literature to explain *why* MILP performs so well on this type of combinatorial discrete choice problem.

2 Single and multi-stage facility location problems

Single-stage facility location problem The classic facility location problem is stated in Box 2.1. This objective is to choose locations ℓ where a number of facilities need to be located in order to minimize total costs paid to serve different clients / markets denoted by n . The cost function has a variable part with unitary cost $c_{\ell n}$ applied to quantity q_n and a fixed cost ϕ_ℓ . Decisions consists of the opening/activation of the plant, y_ℓ , and an allocation of which plant serves n , denoted by $x_{\ell n}$. This is an integer programming problem since we have specified both x and y as binary variables in (4) and (5). Constraints (2) and (3) impose that all clients are served by only one plant, and that the plant must be active to be chosen. The number of production stages is denoted by K , which is set to one here. A tutorial on this problem with Julia code is available at https://jump.dev/JuMP.jl/stable/tutorials/linear/facility_location/.

Box 2.1: Single-stage UFLP: cost-minimization formulation

$$\text{minimize} \quad \sum_{n \in N} \sum_{\ell \in L} c_{\ell n} q_n x_{\ell n} + \sum_{\ell \in L} \phi_\ell y_\ell \quad (1)$$

$$\text{subject to} \quad \sum_{\ell \in L} x_{\ell n} = 1 \quad \forall n \in N \quad (2)$$

$$x_{\ell n} \leq y_\ell \quad \forall \ell \in L, \forall n \in N \quad (3)$$

$$x_{\ell n} \in \{0, 1\} \quad \forall \ell \in L, \forall n \in N \quad (4)$$

$$y_\ell \in \{0, 1\} \quad \forall \ell \in L \quad (5)$$

The cost-minimization formulation is unappealing as an economic model because we

would like q_n to be endogenous and depend on the cost of serving that client n . Fortunately, it is easy to modify the problem to resolve this concern. We just need to make an assumption on variable profits $\pi(c_{\ell n})$. Constant elasticity demand is a natural choice, yielding $\pi(c_{\ell n}) = A_n c_{\ell n}^{1-\eta}$, where A_n is a demand shifter that also incorporates a markup term and η is the price elasticity of demand. Relative to Box 2.1, besides the change in objective function, Box 2.2 has also modified the specification of the assignment variable $x_{\ell n}$, replacing the binary requirement of equation (4) with the non-negative requirement of equation (9). This change is solely for computational reasons: the optimized values of $x_{\ell n}$ will always be binary. In the terminology of the operations research literature, the solved $x_{\ell n}$ are guaranteed to be binary because the FLP is totally unimodular.¹

Box 2.2: Single-stage UFLP: profit maximization formulation

$$\text{maximize} \quad \sum_{n \in N} \sum_{\ell \in L} A_n c_{\ell n}^{1-\eta} x_{\ell n} - \sum_{\ell \in L} \phi_{\ell} y_{\ell} \quad (6)$$

$$\text{subject to} \quad \sum_{\ell \in L} x_{\ell n} \leq 1 \quad \forall n \in N \quad (7)$$

$$x_{\ell n} \leq y_{\ell} \quad \forall \ell \in L, \forall n \in N \quad (8)$$

$$x_{\ell n} \geq 0 \quad \forall \ell \in L, \forall n \in N \quad (9)$$

$$y_{\ell} \in \{0, 1\} \quad \forall \ell \in L \quad (10)$$

A more subtle difference with respect to the cost-min formulation is the profit-max formulation has an inequality in equation (7). This allows for some markets n to not be served from any source. The reason for this is that we do not want to force the firm to serve customers if there is no way to do so profitably. There is no fixed cost of serving a customer in Box 2.2 so if the firm serves any customer, it will serve *all* customers. However, the firm has the option of not serving any customers if the fixed costs of *facility activation* are too high. The problem in Box 2.2 is a mixed integer linear program (MILP), since it mixes x , a non-negative real number, with y , a binary variable.

In order to illustrate how the model works, we parameterize it as follows. First, we set all the demand shocks to be identical ($\ln A_n = \exp(\mu_A) \forall n$). Second, for a single-stage production, the delivered marginal cost function is specified as

$$c_{\ell n} = \exp(\beta \ln D_{\ell n} + u_{\ell n} + v_{\ell}). \quad (11)$$

¹This is important: if the solution actually involved fractional $x_{\ell n}$, the set-up would be problematic because we would have to specify how firms price when they sell goods with different costs in the same market.

The data generating process therefore combines four types of heterogeneity:

1. Plants and consumers are located at points determined by uniform random draws in the unit square. This leads $D_{\ell n}$ being one plus Euclidean distance from ℓ to n .²
2. Path-specific random cost shocks $u_{\ell n}$ distributed $\mathcal{N}(0, \sigma_u)$
3. Site-specific random variable cost shocks v_ℓ distributed $\mathcal{N}(1, \sigma_v)$.³
4. Site-specific random fixed cost shocks, with $\ln \phi_\ell$ distributed $\mathcal{N}(\mu_\phi, \sigma_\phi)$

To make the FLP more concrete, we simulate a two-dimensional world where coordinates of L potential production locations and N markets are drawn randomly from a unit interval. Every production location is differentiated by their realized fixed costs, while having the same marginal cost of production. Consumer markets are identical in both fixed cost of market entry (set to 0 such that all markets are served) and aggregate demand. For the single-stage problem, we draw $L = 30$ and $N = 40$.

When the firm serves all markets in the single-stage problem, a higher trade elasticity increases the number of plants built through the proximity-concentration trade-off. This is shown by the comparison of panels (a) and (b) of Figure 1. More plants closer to the consumer markets are open, facing higher distance costs.

Multi-stage facility location problem The problem stated in Box 2.3 extends Box 2.2 to allow multiple stages of production, including what we may think of as a distribution or market entry stage, in which the firm decides whether to bear the fixed cost, ϕ_n , needed for serving consumer n . Depending on the application, a “consumer” could be an individual, a city, or a country. The core requirement is that a consumer is characterized by a final demand curve and a fixed cost of being served. The firm sets the binary “market entry” variable z_{mn} to one when it is profitable to serve that consumer.

We parameterize the two-stage problem as having the path cost equal to

$$c_{\ell n} = (w_{\ell 2})^{\alpha_{22}} (w_{\ell 1}^{\alpha_{11}} \tau_{\ell 1 \ell 2})^{\alpha_{12}} \tau_{\ell 2 n} \varepsilon_{\ell n}, \quad (12)$$

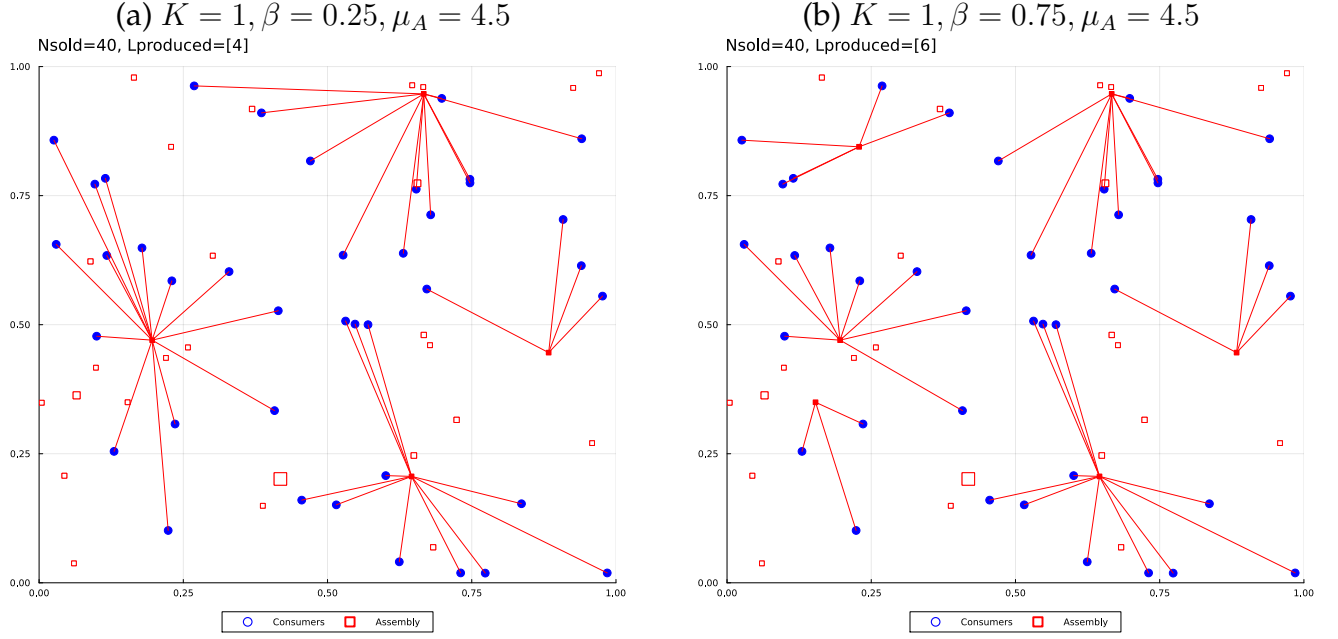
and for a three-stage chain, the path cost is

$$c_{\ell n} = (w_{\ell 3})^{\alpha_{33}} \left[(w_{\ell 2})^{\alpha_{22}} (w_{\ell 1}^{\alpha_{11}} \tau_{\ell 1 \ell 2})^{\alpha_{12}} \tau_{\ell 2 \ell 3} \right]^{\alpha_{23}} \tau_{\ell 3 n} \varepsilon_{\ell n}. \quad (13)$$

²We add one to the Euclidean distance to avoid negative log distances. One can interpret the internal distance within a location is one.

³This is equivalent to a log-normal distribution for the z_ℓ productivity shock in Arkolakis et al. (2023)

Figure 1: Single-stage simulation by trade elasticity



Notes: Other parameters are $L = 30, N = 40, \sigma_v = 0, \sigma_u = 0, \sigma_\phi = 1.5, \mu_\phi = 0, \text{seed} = 0$. Size of dot indicates the level of fixed costs. For example, larger triangle implies the fixed cost of building a battery facility is higher.

Generalizing the number of stages, the path cost for a multi-stage chain is

$$c_{\ell n} = \prod_{k=1}^K (w_{\ell^k})^{\alpha_{k,k} \gamma_k} \times \prod_{k=1}^{K-1} \tau_{\ell^k \ell^{k+1}}^{\gamma_k} \times \tau_{\ell^K n} \times \varepsilon_{\ell n} \quad (14)$$

$$= \exp \left(\sum_{k=1}^{K-1} \gamma_k (\alpha_{k,k} v_{\ell^k} + \beta \ln D_{\ell^k \ell^{k+1}}) + \alpha_{K,K} v_{\ell^K} + \beta \ln D_{\ell^K n} + u_{\ell n} \right), \quad (15)$$

where $\gamma_k \equiv \prod_{i=k}^{K-1} \alpha_{i,i+1}$, and $w_{\ell^k} = \exp(v_{\ell^k})$, and $\tau_{\ell^k \ell^{k+1}} = \exp(\beta \ln D_{\ell^k \ell^{k+1}})$, and $\varepsilon_{\ell n} = \exp(u_{\ell n})$. While we assume the same distance costs, β , apply at all stages, this is only to reduce the number of parameters. In realistic applications, distance costs are likely to differ substantially across stages of production.

For notational brevity we have specified costs for a representative product above. In Box 2.3 we generalize to allow for M products, indexed by m .

Box 2.3: Multi-product Multi-stage UFLP

$$\text{maximize} \quad \sum_{m \in M} \sum_{n \in N} \sum_{\ell \in \mathcal{L}} A_n c_{\ell mn}^{1-\eta} x_{\ell mn} - \sum_{k=1}^K \sum_{\ell \in L^k} y_{\ell}^k \phi_{\ell}^k - z_{mn} \phi_n \quad (16)$$

$$\text{subject to} \quad \sum_{\ell \in \mathcal{L}} x_{\ell mn} \leq z_{mn} \quad \forall m, n \quad (17)$$

$$\sum_{\ell^{-k} \in \mathcal{L}^{-k}} x_{\ell mn} \leq y_{\ell}^k \quad \forall m, n, k, \ell \in L^k \quad (18)$$

$$x_{\ell mn} \geq 0 \quad \forall m, n, \ell \quad (19)$$

$$y_{\ell}^k \in \{0, 1\} \quad \forall m, n, k, \ell \in L^k \quad (20)$$

$$z_{mn} \in \{0, 1\} \quad \forall m, n \quad (21)$$

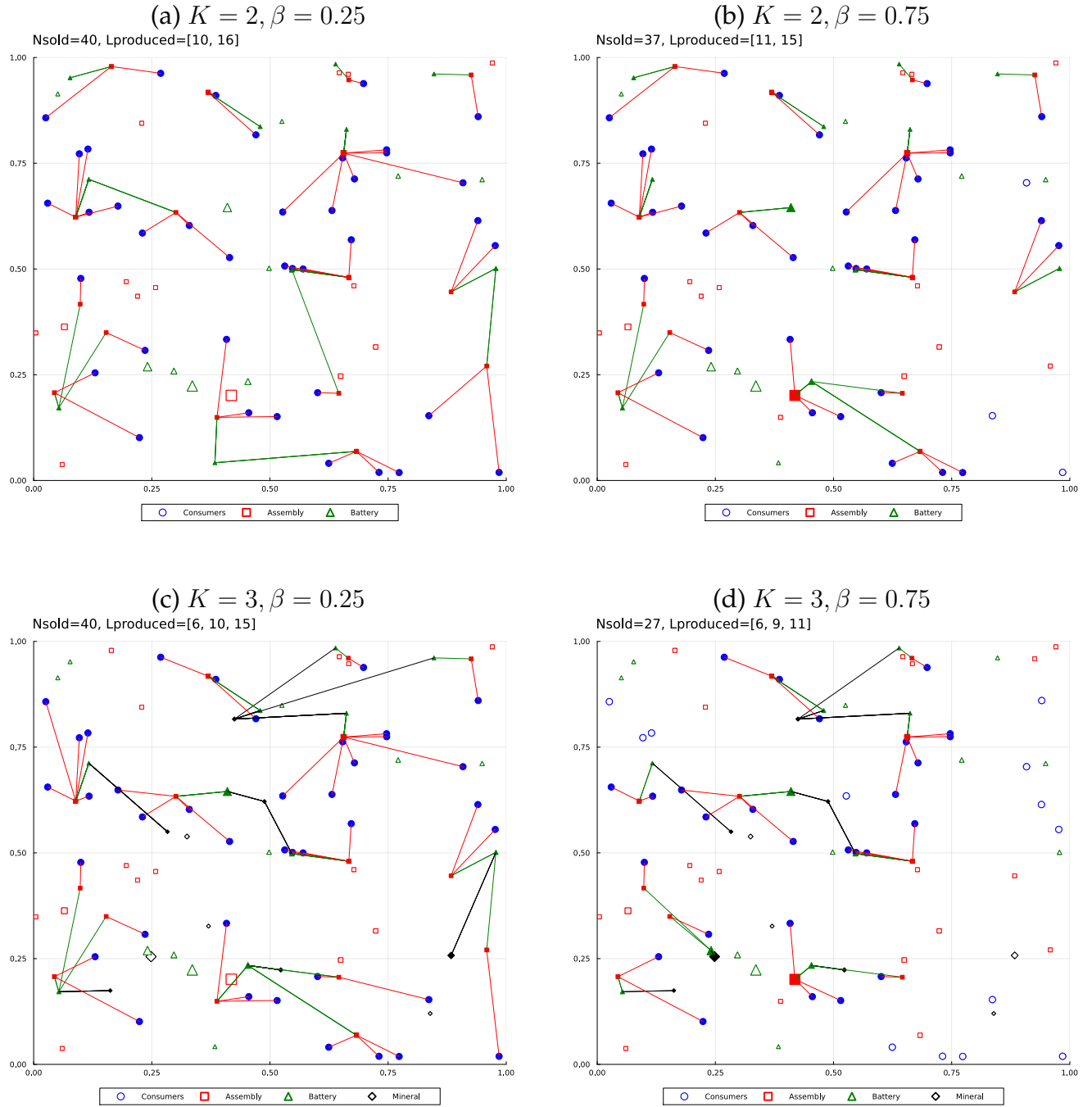
$$\text{where} \quad \ell \equiv \{\ell^1, \ell^2, \dots, \ell^K\} \quad \forall \ell^k \in L^k \quad (22)$$

$$\ell^{-k} \equiv \{\ell^1, \dots, \ell^{k-1}, \ell^{k+1}, \dots, \ell^K\} \quad \forall \ell^i \in L^i \text{ and } i \neq k \quad (23)$$

Once we allow market entry to be endogenous, there are two opposing forces with respect to increases in trade elasticity. One is that higher trade costs will lead to more plants being built closer to consumers. The other is that higher trade costs along the supply chain makes entry unprofitable in some markets, and fewer consumers served reduces the number of plants.

We illustrate the multi-stage FLP in Figure 2. For $K = 2$, the number of potential locations is $L = [20, 30]$; for $K = 3$, it is $L = [10, 20, 30]$. The number of markets remains at 40. Each firm produces and sells a single product. Comparing Panel (b) to Panel (a) in Figure 2, we observe that the endogenous loss of consumer markets dominates the proximity force in the downstream (assembly) stage, and thus one assembly plant is dropped. The reverse happens in the upstream (battery) stage where one plant is added. Despite the small change in the number of plants, their spatial allocation is re-organized. Comparing Panel (b) to Panel (a) demonstrates a trade-off between variable cost and fixed cost. Specifically, a nearby high fixed cost assembly plant (red square in middle south) and a nearby high fixed cost battery plant (green triangle in middle north) are open when trade costs along paths increase and lowering trade costs dominates saving of the fixed cost. A similar pattern is observed when comparing Panel (d) to Panel (c). Figure 2 also shows that the effect of trade costs is magnified the longer is the supply chain. The number of served consumers drops by 13 and the total number of plants drops by five when $K = 3$, whereas the reduction is three and zero respectively when $K = 2$.

Figure 2: GVC simulation by stages and trade elasticity



Notes: In the case of $K = 2$, number of locations is $L = \{20, 30\}$; when $K = 3$, $L = \{10, 20, 30\}$. Other parameters are $N = 40$, $\sigma_v = 0$, $\sigma_u = 0$, $\sigma_\phi = 1.5$, $\mu_\phi = 0$, $\phi_n = 1$, $\mu_A = 9.5$, seed = 0. Size of dot indicates the level of fixed costs. For example, larger triangle implies the fixed cost of building a battery facility is higher. Fixed cost is proportionally lower when having more stages of production.

Now that we have stated the different versions of the problem, we turn to a discussion of various aspects of their solutions.

3 Global solution

A familiar problem confronting non-linear optimization problems is that algorithm may exit due to convergence at local optimum that is substantially inferior to the global optimum. This is not the case with mixed integer programming. The commercial solver we use, Gurobi, guarantees a global optimum when it concludes with the flag “OPTIMAL”.⁴

Gurobi, employs a variety of techniques to solve MILPs that make it substantially faster than the best open source MILP solver, HiGHS.⁵ Our main interest here is to explain how the algorithm can guarantee a global optimum. The central method is *linear programming based branch and bound* (LPB&B). Understanding LPB&B requires a specialized vocabulary that is not fully standardized across references. The treatment here is a synthesis of Gurobi documentation⁶, Conforti et al. (2014, p. 7), and Nemhauser and Wolsey (1999, pp. 352–356).

The “search tree” shown in Figure 3—adapted from Figure 1.4 in (Nemhauser and Wolsey, 1999, p. 353)—provides a useful illustration. This problem has three binary variables, y_1, y_2, y_3 . These could be whether a firm puts a plant in Asia, North America, or Europe, for example. There are 2^L possible configurations (000, 001, ..., 111 for $L = 3$). For such small n , one would just evaluate the profit of each y vector, $\pi(y)$, and chose the best one. This becomes impractical once L is much over 20 for the single stage problem.

LPB&B proceeds as follows. Start at the top node S . Solve the *linear relaxation* of the full problem. The linear relaxation is the linear programming version of the integer programming problem: it treats the y_1, y_2 and y_3 as continuous variables and just maximizes profits subject to any set of linear constraints. Thus, it *relaxes* constraints (20).⁷ The resulting solution is y_R and the resulting profit is π_R .⁸ The relaxation achieves weakly higher profits than the solution to the MILP because it imposes fewer constraints. If it so happens that all the y_R are binary (no fractional solutions), then y_R is the optimal solution to the

⁴<https://docs.gurobi.com/projects/optimizer/en/current/reference/numericcodes/statuscodes.html>

⁵Gurobi offers a free academic license, but HiGHS is still useful for smaller problems.

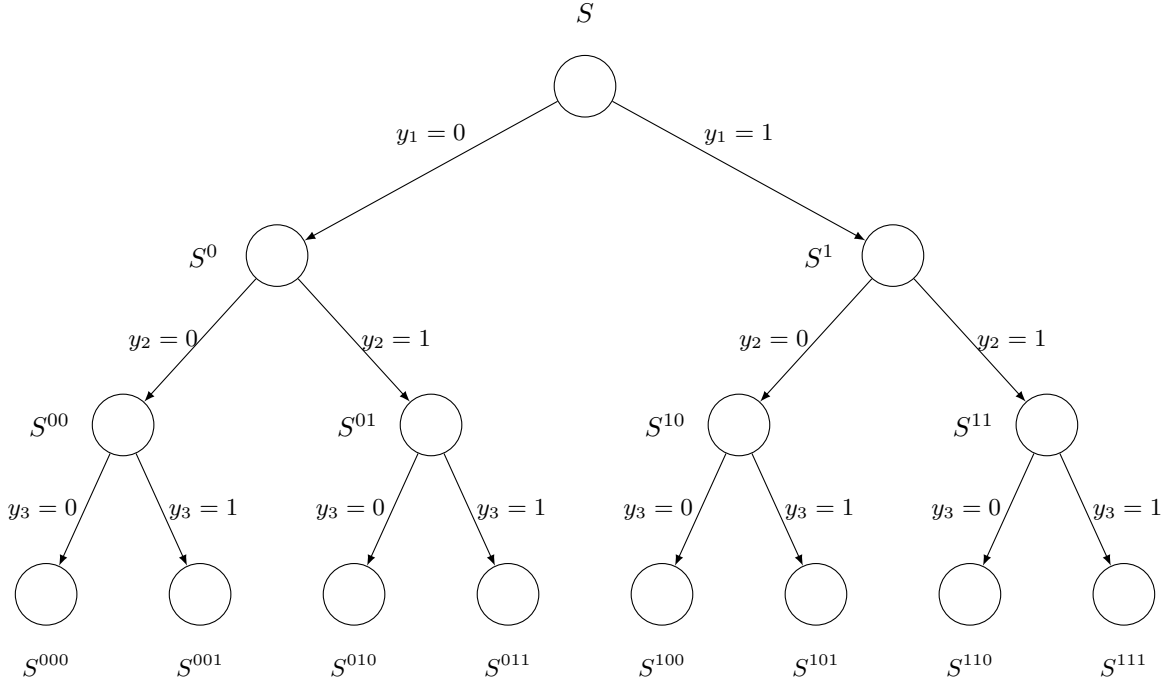
⁶<https://www.gurobi.com/resources/mixed-integer-programming-mip-a-primer-on-the-basics/>

⁷We omit the endogenous entry decision, constraint (21), in this illustration to keep the branch and bound tree of a manageable size.

⁸Each LP can be expressed in primal and dual forms. The Strong Duality theorem (Lee, 2004, p. 15) establishes that at the optimum the primal and dual solutions are identical. This fact provides a means to certify that the LP algorithm has arrived at the global optimum.

MILP. The algorithm can declare victory. There is no way to do better than π_R because the solution to a linear programming problem (when it exists) is always a global optimum.

Figure 3: Branch and Bound



If the linear relaxation of S yields a y_{LR} with fractional elements, then the program starts to search by moving down through the nodes shown in the tree in figure 3. At each step, there are two variables to track and potentially update: The *incumbent* has profit π_I , and the *best bound* has profit π_B . The best bound establishes an upper limit for profits that cannot be exceeded. It starts out as $\pi_B = \pi_R$ of the root node (the complete problem S). We update the best bound as the maximum profit of all the current leaf nodes. The incumbent could be the profit of a solution generated through a heuristic method, that is one that obtains a solution that meets the constraints but has no claim to optimality. Alternatively, we can initialize π_I at $-\infty$ and wait until the first binary solution to a sub-problem in the search tree. Once we have values for π_I and π_B , we compute the gap $= (\pi_B - \pi_I)/\pi_B$. As each node in the tree gets processed, π_I will be updated upwards (finding more profitable solutions). Meanwhile, π_B will be updated *downwards* as the best bounds are tightened by confining more of the y_k to binary values. The search concludes with a global optimum when the gap is brought below a tolerance level.

Branch and bound routines are guaranteed to find the optimum, so the real challenge

is to process as few nodes as possible (otherwise we end up doing brute force). The way LPB&B reduces the number of nodes to explore is by *fathoming* (in the Gurobi documentation), also referred to as *pruning* in many textbook treatments. We illustrate how this works in Table 1.

Table 1: Processing and pruning of node S^i in the search tree

Case	Outcome of LP relaxation	Action to take
1	No LP_i solution satisfies constraints	Prune by <i>infeasibility</i>
2	LP^i solution's $\pi_R^i < \pi^I$	Prune by <i>bound</i> (value dominance)
3	LP^i solved y^i is all binary	Prune by <i>integrality</i> , if $\pi_R^i < \pi_I$, update π_I
4	LP^i y^i has fractional values, $\pi_R^i > \pi_I$	Update π_B , search sub-nodes

In the first three cases, we say *the tree is pruned at node i* or that node has been fathomed. It becomes a permanent “leaf” in the search tree. Since only case 4 requires further downward search, LPB&B is fast when this case is rare.

Branch and bound search guarantees a global optimum will be reached, by brute force if necessary. However, good MILP solvers employ a large array of “tricks” to speed the process while still guaranteeing global optimality. The tricks include heuristic algorithms and the use of carefully selected *cutting planes* to reduce the solution space without removing any potential optima.⁹

4 Computational performance of MILP

In this section, we evaluate the computational performance of the mixed integer linear programming solution to combinatorial discrete choice problems using a set of simulations.¹⁰ We conduct three different speed tests, each highlighting different aspects of the algorithm.

4.1 Single-stage UFLP compared with AES squeezing algorithm

While MILP is an old method, well established for solving plant location problems for over 50 years, most trade economists are more familiar with the methods that rely on supermodularity of the profit function, introduced in economics by Jia (2008) and adapted to international sourcing problems by Antràs et al. (2017). Like MILP, the Jia method has

⁹Papadimitriou and Steiglitz (1998) provides an intuitive illustration of how this method works.

¹⁰Computational performance tests are run on one of the author’s MacBook Pro with Apple M2 Max chip and 64GB memory via serial processing.

deep roots in the mathematics of optimization literature; she cites theorems by Tarski (1955) and Topkis (1978). More recently, Arkolakis et al. (2023) extended the method to handle submodular functions. The authors provide a nice overview of the key insight:

“[P]ositive complementarities induce a form of *choice monotonicity* that can be used to iteratively discard suboptimal choice combinations without computing their payoff. If an option yields a negative payoff even when all other options are selected—maximizing complementarities—it cannot be part of the optimal combination. Likewise, if an option has a positive payoff even in isolation, it must be included.”

In Appendix A we show how squeezing methods can dramatically reduce the number of times profits must be evaluated. However, like MILP, the AES method does not have guaranteed properties. Therefore, it is useful to compare their performance in practice.

To make a fair comparison between MILP and squeezing methods (which we will abbreviate as AES going forward), we need a problem that can be solved using both methods. The AES method is not generally applicable to multi-stage problems and MILP requires a linear objective function and constraints. We therefore investigate the case of a single-stage uncapacitated facility location problem (UFLP). This problem can be solved by both methods. We derive this problem as a special case of the problem considered by Arkolakis et al. (2023).

The firm’s variable profit selling to market n is a function of its set of potential sourcing locations, denoted $\mathcal{L}$. With iso-elastic demand proportional to $p^{-\eta}$, the variable profit in market n is $\pi_n(\mathcal{L}) = A_n c_n(\mathcal{L})^{1-\eta}$, where $\mu = \eta/(\eta - 1)$ is the optimal markup with A_n incorporating $(\eta - 1)^{\eta-1} \eta^{-\eta}$. The firm chooses $\mathcal{L} \subset L$ to maximize profit net of fixed costs ϕ incurred in each activated plant:

$$\mathcal{L}^* = \arg \max_{\mathcal{L}} \left\{ \sum_n \pi_n(\mathcal{L}) - \sum_{\ell \in \mathcal{L}} \phi_\ell \right\}.$$

With a continuum of inputs as in Tintelnot (2017), the cost function is

$$c_n(\mathcal{L}) = \left[\sum_{\ell \in \mathcal{L}} c_{\ell n}^{-\theta} \right]^{-1/\theta}, \quad \text{where} \quad c_{\ell n} = w_\ell \tau_{\ell n} / z_\ell. \quad (24)$$

Corresponding to our earlier cost setup in equation (11), $v_\ell \equiv \ln(w_\ell/z_\ell)$ and $\ln \tau_{\ell n} \equiv \beta \ln D_{\ell n} + u_{\ell n}$.

We can re-express the AES problem, replacing the $\mathcal{L}$ set notation with the x and y

binary variables to make it easier to compare with the FLP shown in Box 2.3.

Box 4.1: AES problem with x and y

$$\text{maximize} \quad \sum_{n \in N} A_n \left[\sum_{\ell \in L} c_{\ell n}^{-\theta} x_{\ell n} \right]^{-(1-\eta)/\theta} - \sum_{\ell \in L} \phi_{\ell} y_{\ell} \quad (25)$$

$$\text{subject to} \quad x_{\ell n} \leq y_{\ell} \quad \forall \ell \in L, \forall n \in N \quad (26)$$

$$x_{\ell n} \in \{0, 1\} \quad \forall \ell \in L, \forall n \in N \quad (27)$$

$$y_{\ell} \in \{0, 1\} \quad \forall \ell \in L \quad (28)$$

The fixed costs in this model are associated with activating particular production locations ($y_{\ell} = 1$) but destination n does not pay a fixed cost to use each source country. Hence, for finite θ the cost function in equation (24) implies that each n will use *all* the ℓ that are active. Taking this into account, we can simplify the statement of the AES problem by eliminating the $x_{\ell n}$ variables. This gives rise to Box 4.2.

Box 4.2: AES problem with y only

$$\text{maximize} \quad \sum_{n \in N} A_n \left[\sum_{\ell \in L} c_{\ell n}^{-\theta} y_{\ell} \right]^{-(1-\eta)/\theta} - \sum_{\ell \in L} \phi_{\ell} y_{\ell} \quad (29)$$

$$\text{subject to} \quad y_{\ell} \in \{0, 1\} \quad \forall \ell \in L \quad (30)$$

For the limiting case as $\theta \rightarrow \infty$, we prove in appendix B that Box 4.2 is equivalent to Box 2.2, the uncapacitated facility location problem formulation, for a *single* stage of production. The comparable setting between MILP and AES is a single-stage combinatorial discrete choice problem by a single-product firm whose market entry is exogenous. The AES method converges on this problem as their $\theta \rightarrow \infty$, that is, as the different location choices become perfect substitutes. In that limiting case, we have $c_n(\mathcal{L}) = \min_{\ell \in \mathcal{L}} c_{\ell n}$. Perfect substitution implies single-sourcing from the least-cost $\ell \in \mathcal{L}$. Randomness arises from firm-location productivity shocks (z_{ℓ}) and the locations of producers and consumers, both drawn from a uniform distribution on a unit square. The Euclidean distances between locations determine $\tau_{\ell n}$.

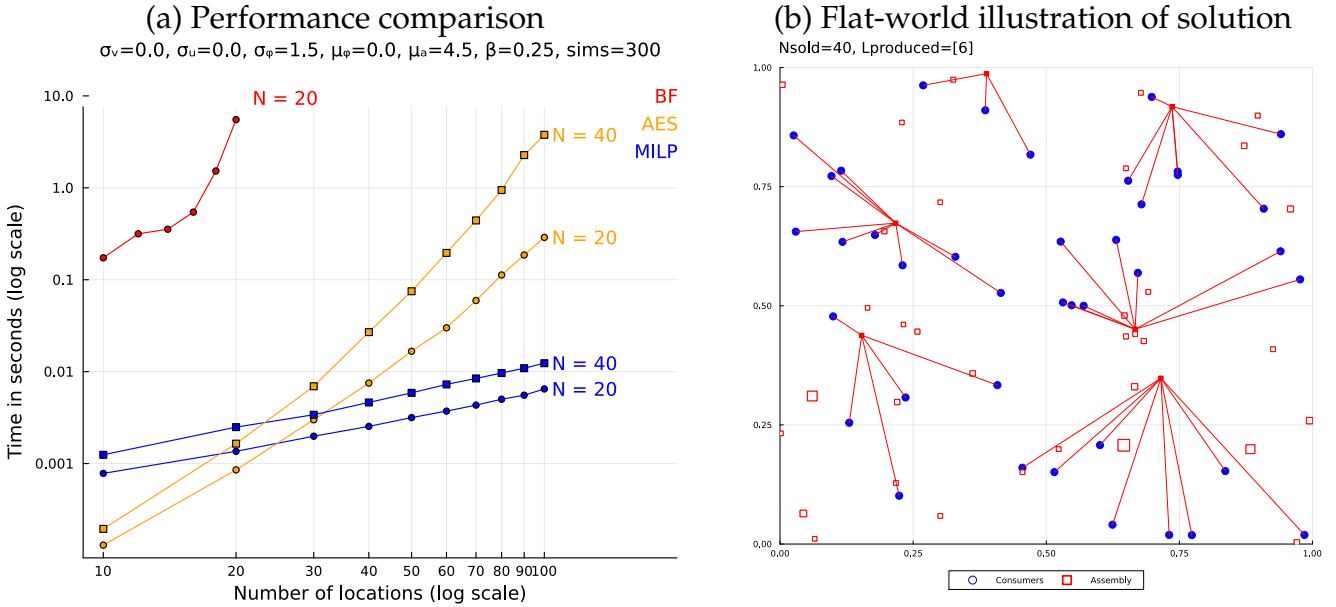
In the first test, we compare how long it takes to solve for the optimal set of facility locations comparing three methods:

1. formulating firm's total profit as mixed integer linear programming and solving with Gurobi (MILP)

2. using the squeezing algorithm proposed by Arkolakis et al. (2023) and implemented in the Julia CDCP (AES)
3. “brute-force”: computing the profit associated with every possible configuration, then sorting to find the maximum profit configuration (BF)

We generate data by randomly drawing L potential production locations and N markets on a unit square. The firm is assumed to sell its product in every market. The number of locations L varies within $\{10, 20, 30, 40, 50, 60, 70, 80, 90, 100\}$, and the number of markets varies within $\{20, 40\}$. We found that brute-force demands too much memory to be applied for more than 20 locations and markets.¹¹ For brute-force, we restrict $L \in \{10, 12, 14, 16, 18, 20\}$ and $N = 20$. The data is simulated 300 times to compute the average algorithm’s run time.

Figure 4: Three ways to solve a single-stage UFLP



Notes: Flat-world illustration uses $L = N = 40$ and seed = 0. Panel (b) uses the parameter setting as in Panel (a).

Figure 4 Panel (a) shows that the run times (seconds) for MILP and AES are comparable at low number of potential locations, with AES being slightly faster. As the number of locations increases, the computational performance between these two algorithms start to diverge and MILP dominates. The reason is that for a single-stage uncapacitated facility location problem, the possible assignment for each market is L , and thus the computation

¹¹The memory requirement is proportional to 2^L , so we can project the memory needed for $L = 25$ to be almost 200GB.

time of MILP increases with locations at a linear rate (conditional on the number of markets). Nevertheless, both algorithms drastically reduce the run time compared to brute force, and extend our capability of solving combinatorial discrete choice problem in real world applications. Panel (a) in Table 3 presents the detailed runtime, and corroborates the unique global solution of MILP in Section 3 which is identical to the solution via AES.

With the parameter setting in Panel (b) of Figure 4, when all 40 consumers are served, there are six plants open. Figure 5 provides more details of the average run time comparison for each parameter setting, and their respective number of plant openings.

There are four notable features in Figure 5. The first is that AES is very fast whenever there are few locations left undetermined (gray) after the initial squeezing. In case (a) the number of undetermined locations falls to zero quickly with just a little site-cost heterogeneity. Path-level (ℓn) heterogeneity used in case (b) also quickly reduces undetermined locations. In this case it does so by *increasing* the number of definitely active plants. Case (c) introduces fixed cost heterogeneity and this leaves many plants undetermined after initial squeezing. It also leads to more plants in equilibrium (rising from just 1.2 to 5.7 on average). These conflicting effects lead to a hump-shaped time performance for AES. The high distance costs in case (d) also have non-monotonic effects on the performance of AES. The fall in the number of undetermined-after-squeezing locations helps but the rise in the equilibrium number of activated plants hurts. A string feature of figure 5 is how little variation there is in the time taken by Gurobi to solve the MILP. Parameter settings just do not seem to matter much.

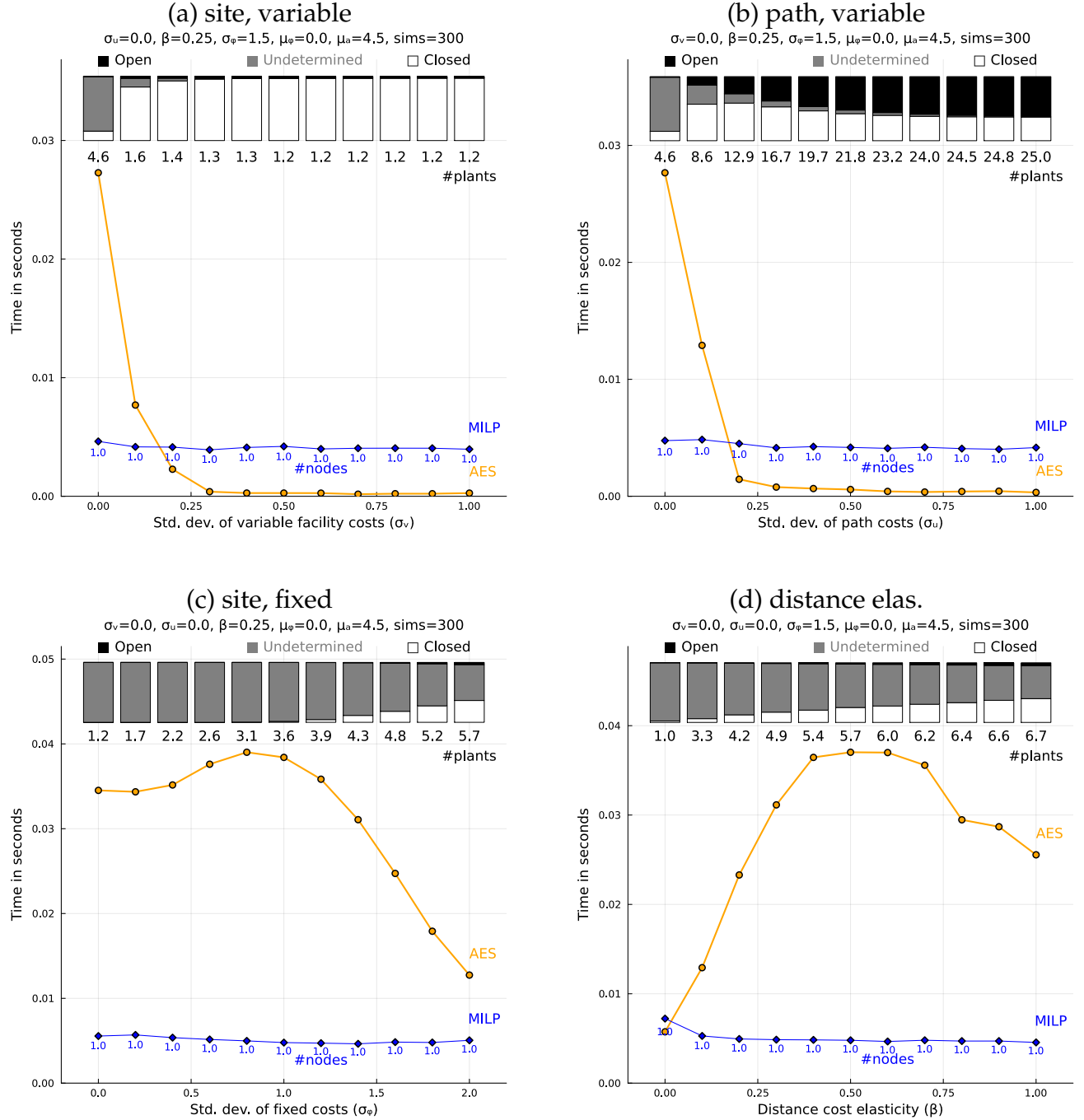
One key takeaway from Figure 5 is that AES is unbeatable when squeezing eliminates the undetermined locations (as in cases a and b) but there are settings where MILP beats AES. All the times in this chart for the 40-location case are under 0.05 seconds on average: Both solution methods are very fast for moderate-sized problems.

Table 2 focuses on the two cost heterogeneity parameters that cause a monotonic change in AES performance. We compute a single cutoff for when AES beats MILP times. The critical values are very low for $L = 20$ and $L = 40$. However, they increase for 80 locations. This is in line with expectations from Figure 4, where we see AES times rising with the number of locations at a faster rate than MILP.

4.2 Times for multi-stage UFLP

In this section, we evaluate the computational speed solving a two-stage supply chain for MILP alone because introducing more than one stage of production violates the single crossing difference condition required to use AES. We also relax other restrictions com-

Figure 5: How heterogeneity affects relative performance of MILP and squeezing



Notes: $L = N = 40$. We vary each parameter while keeping the rest fixed. For figures corresponding to other L values, please refer to appendix figure C2 and C3.

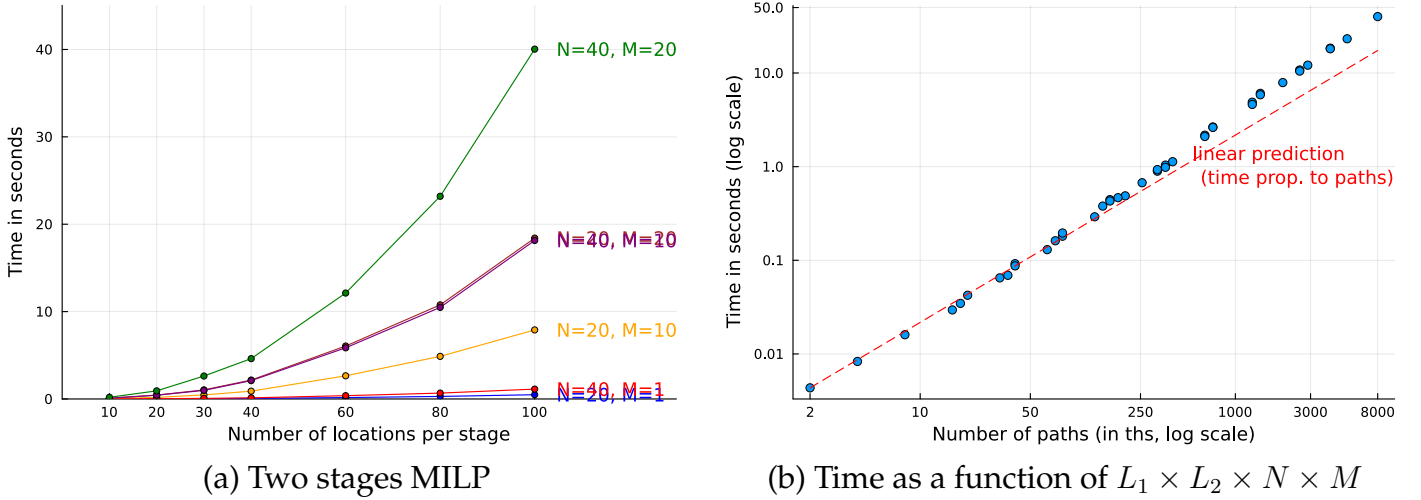
Table 2: Critical parameter values in comparison

Parameter type	Locations (L)		
	20	40	80
Variable facility costs ($\sigma^v \in \{0, 1\}$)	0.00	0.13	0.44
Path costs ($\sigma^u \in \{0, 1\}$)	0.00	0.16	0.49

Each cell reports the minimum parameter value at which the squeezing method becomes faster than MILP, given that run-time of the squeezing method changes monotonically with respect to the parameter value, and hence σ_v and σ_u only as shown in Figure 5. A value at the boundary of the parameter range indicates that one algorithm dominates throughout. There are $N = 40$ consumers, and 300 number of simulations for each setup. We vary each parameter while keeping others fixed to $\sigma_v = 0, \sigma_u = 0, \sigma_\phi = 1.5, \mu_\phi = 0, \mu_A = 4.5, \beta = 0.25$.

pared to the first test by allowing for multi-product firm, represented by $M > 1$, and endogenous market entry. We vary the number of locations $L \in \{10, 20, 30, 40, 60, 80, 100\}$, the number of markets $N \in \{20, 40\}$, and the number of products $M \in \{1, 10, 20\}$, and simulate 100 repetitions.

Figure 6: Computational Performance of MILP



Notes: Parameters are set to $\phi_n = 1, \sigma_v = 0, \sigma_u = 0, \sigma_\phi = 1.5, \mu_\phi = 0, \mu_A = 9.5, \beta = 0.25$. The performance is averaged across 100 simulations.

When $K = 2$, there are L^2 potential assignments for each product-market. For this reason time is convex in the number of locations in Panel (a) of Figure 6. Moreover, because the number of products and the number of markets are isomorphic in determining the

size of the problem, only the product $N \times M$ matters and we observe almost equal computation time for $N = 40, M = 10$ and $N = 20, M = 20$.¹² Panel (b) of Table 3 shows how the time scales with the size of the problem. The “size” can be thought of as the number of variables and the number of constraints. However, it turns out that the number of paths is what varies the most. The plot in panel (b) of Figure 6 shows that time is very close to linear in $L_1 \times L_2 \times N \times M$. The log-log regression has a coefficient of 1.1, where 1.0 would correspond to perfect linearity.

In the last test, we add one more stage in the supply chain by having $K = 3$. The number of potential assignments for each product-market increases to L^3 . Therefore, we limit the potential locations to 30. The run time increases more for each location increments in Panel (c) compared to Panel (b) in Table 3.

5 Why MILP is fast

There are two reasons behind the strong performance of mixed integer linear programming to solve for the high-dimensional combinatorial discrete choice problem. First, the LP relaxation very often yields integral solutions, which avoids the problem of long branch-and-bound searches. Second the linear programming relaxation is itself very fast because the FLP fits within a broad class of problems where the simplex method is known to be able to solve for the optimum with a number of steps that is bounded by the product of the number of locations and the number of customers. That is instead of the number of steps being exponential in L , it just depends on the product LN . We dig into these points in the following two subsections.

5.1 The LP relaxation of MUFLP is (usually) binary

The surprising feature of our MUFLP is that not only is sub-node search rare, it usually isn’t required at all. That is, in an overwhelming majority of case the LP relaxation of the full problem (the root node S in figure 3) is binary. The first published paper noting this surprising fact for a facility location problem was ReVelle and Swain (1970). It was further explored by Cornuejols et al. (1983), who write “for reasons that are barely understood, very frequently, the [LP relaxation] has an optimal integral solution...Thus an efficient method for solving the [LP relaxation] will also be an efficient method for solving many instances of the [uncapacitated facility location] UFL Problem.”

¹²The slight difference between column (3) and column (5) shown in Panel (a) of Table 3 is due to simulation error.

Table 3: Average computation time in seconds

Panel (a): $K = 1$, MILP and AES

L_k	$N = 20$			$N = 40$		
	MILP	AES	$\ \mathcal{L}_{\text{MILP}}^* - \mathcal{L}_{\text{AES}}^*\ _1$	MILP	AES	$\ \mathcal{L}_{\text{MILP}}^* - \mathcal{L}_{\text{AES}}^*\ _1$
10	0.0007	0.0002	0.0	0.0013	0.0002	0.0
20	0.0014	0.0008	0.0	0.0024	0.0016	0.0
30	0.0020	0.0029	0.0	0.0034	0.0069	0.0
40	0.0026	0.0074	0.0	0.0046	0.0274	0.0
50	0.0031	0.0166	0.0	0.0057	0.0739	0.0
60	0.0037	0.0311	0.0	0.0072	0.1969	0.0
70	0.0044	0.0614	0.0	0.0084	0.4459	0.0
80	0.0050	0.1178	0.0	0.0095	0.9620	0.0

Panel (b): $K = 2$, MILP

L_k	$N = 20$			$N = 40$		
	$M = 1$	$M = 10$	$M = 20$	$M = 1$	$M = 10$	$M = 20$
10	0.00	0.04	0.09	0.01	0.09	0.20
20	0.02	0.18	0.44	0.03	0.43	0.93
30	0.03	0.47	1.04	0.07	0.98	2.63
40	0.06	0.89	2.17	0.13	2.10	4.62
60	0.16	2.65	6.06	0.38	5.86	12.12
80	0.29	4.88	10.77	0.67	10.48	23.20
100	0.49	7.90	18.41	1.13	18.14	40.04

Panel (c): $K = 3$, MILP

L_k	$N = 20$			$N = 40$		
	$M = 1$	$M = 10$	$M = 20$	$M = 1$	$M = 10$	$M = 20$
10	0.04	0.54	1.23	0.09	1.16	2.99
20	1.85	18.36	39.91	3.77	42.78	98.08
30	7.58	73.71	158.07	15.86	170.80	283.46

Notes: For panel (a), parameters are set to $\phi_n = 0, \sigma_v = 0, \sigma_u = 0, \sigma_\phi = 1.5, \mu_\phi = 0, \mu_A = 4.5, \beta = 0.25$. The performance is averaged across 300 simulations. For panel (b) and (c), parameters are set to $\phi_n = 1, \sigma_v = 0, \sigma_u = 0, \sigma_\phi = 1.5, \mu_\phi = 0, \mu_A = 9.5, \beta = 0.25$. The performance is averaged across 100 simulations.

Figure 7: High success rate of linear relaxation

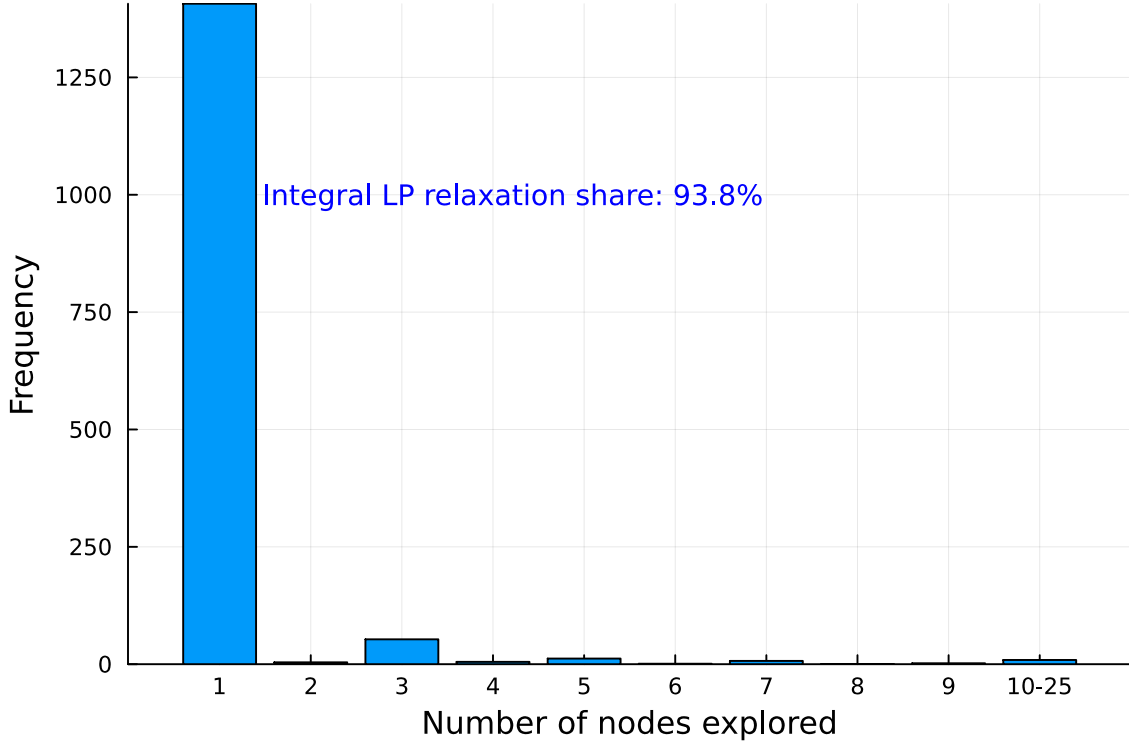


Figure 7 illustrates this puzzling regularity for the facility location problem that we use in Head et al. (2026) within a simulated method of moments estimation of structural parameters. For each iteration in the parameters, the procedure solves the two-stage facility location problem for 15 firms and 100 draws of fixed and variable cost shocks each. Across these 1500 simulations for the final set of estimates, we see that in 1408 cases, the linear relaxation delivers binary outcomes for all the x and y . For the remaining instances, there are usually fewer than 10 nodes explored during branch and bound. Only one case required 25 nodes. The potential number of nodes was vast.

For the two-stage flat-world simulation setup in Figure 6 (corresponding to Panel (b) in Table 3), we also find only one out of 4200 simulations explore three nodes, and the rest only require a single node.

5.2 The simplex method and the Hirsch conjecture

Having established that the LP relaxation is very often integral (or binary in the case of the FLP), we now want to understand why the LP relaxation itself is (usually) so fast. The feasible region of a linear program is a convex polyhedron. Since the objective function is linear, solutions to LPs are always at the vertices of the feasible region. The Simplex

method progress from one vertex to another through a series of “pivot” operations each of which improves the objective function. The Simplex method is guaranteed to terminate in a finite number of steps because the number of vertices is finite. Unfortunately, there is no guarantee that the number of steps will be small.¹³ Fortunately, Borgwardt et al. (2018) prove that network-flow problems satisfy the 1957 Hirsch conjecture, first published in (Dantzig, 1963, p. 168), which states that the maximum number of steps should be given by the number of facets, f minus the diameter d of the polyhedron. Facets refer to constraints. There are $2LN + 2L + N$ constraints comprising N demand inequalities, LN activity constraints, LN non-negativity constraints on the x , L constraints for $y \geq 0$, and also L for $y \leq 1$. The dimension is just the number of variables: there are L for y and LN for x . Thus $f - d = LN + L + N$. Hence the maximum number of steps rises roughly in proportion to LN as we showed in practice in Figure 6(b). The FLP is not strictly speaking a network flow problem but the linear relaxation shares some key features with network flow problems which is why we suspect that this result holds for the FLP and the MUFLP as well.

6 Conclusion

Conclusion to be added

References

- Antràs, P., T. C. Fort, and F. Tintelnot (2017). The margins of global sourcing: Theory and evidence from US firms. *American Economic Review* 107(9), 2514–64.
- Arkolakis, C., F. Eckert, and R. Shi (2023). Combinatorial discrete choice: A quantitative model of multinational location decisions. Working Paper 31877, National Bureau of Economic Research.
- Black, A. E., J. A. De Loera, N. Lütjeharms, and R. Sanyal (2023). The polyhedral geometry of pivot rules and monotone paths. *SIAM Journal on Applied Algebra and Geometry* 7(3), 623–650.
- Borgwardt, S., J. A. De Loera, and E. Finhold (2018). The diameters of network-flow polytopes satisfy the hirsch conjecture. *Mathematical Programming* 171(1), 283–309.

¹³Black et al. (2023).

- Conforti, M., G. Cornuéjols, and G. Zambelli (2014). *Integer Programming Models*. Springer.
- Cornuejols, G., G. L. Nemhauser, and L. A. Wolsey (1983). The uncapacitated facility location problem. Technical report, Cornell University.
- Dantzig, G. B. (1963). *Linear programming and extensions*. Princeton university press.
- Head, K., T. Mayer, M. Melitz, and C. Yang (2026, February). Industrial policies for multi-stage production: The battle for battery-powered vehicles. Working Paper 34884, National Bureau of Economic Research.
- Jia, P. (2008). What happens when Wal-Mart comes to town: An empirical analysis of the discount retailing industry. *Econometrica* 76(6), 1263–1316.
- Lee, J. (2004). *A first course in combinatorial optimization*, Volume 36. Cambridge University Press.
- Nemhauser, G. L. and L. A. Wolsey (1999). *Integer and Combinatorial Optimization*. John Wiley & Sons.
- Papadimitriou, C. H. and K. Steiglitz (1998). *Combinatorial optimization: algorithms and complexity*. Courier Corporation.
- ReVelle, C. and R. Swain (1970). Central facilities location. *Geographical Analysis* (2), 30–42.
- Tarski, A. (1955). A lattice-theoretical fixpoint theorem and its applications. *Pacific Journal of Mathematics* 5(2), 285–309.
- Tintelnot, F. (2017). Global production with export platforms. *The Quarterly Journal of Economics* 132(1), 157–209.
- Topkis, D. M. (1978). Minimizing a submodular function on a lattice. *Operations Research* 26(2), 305–321.

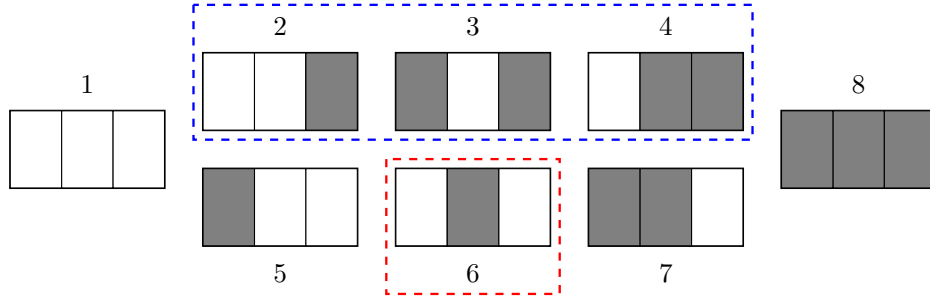
Appendix

A Simple examples of “squeezing” methods

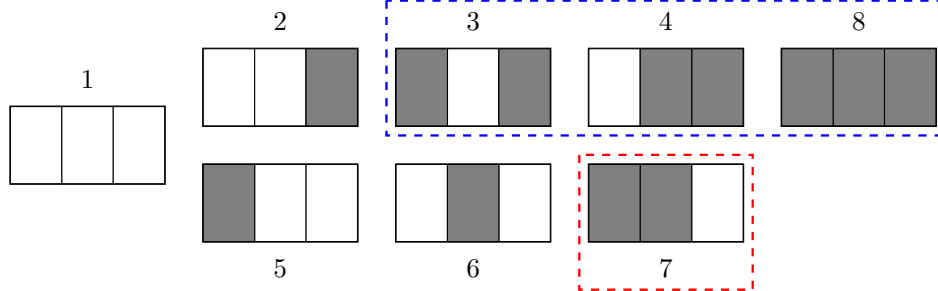
Figure A1 illustrates with the same three-location example we explored in the MILP branch and bound tree in Figure 3.

Figure A1: Three-region illustration of squeezing method

Case (a) supermodular profits (investments are complementary)



Case (b) submodular profits (investments substitute for each other)



In case (a) we start by evaluating configurations 7 and 8: If $\pi_8 < \pi_7$ then we can set position $y_3^i = 0$ for all i . Because of supermodularity, there is no need to evaluate $i = 2, 3, 4$, the configurations contained in the blue dashed rectangle. We still must evaluate 1 and 5. If $\pi_1 < \pi_5$ then position $y_1^i = 1$ for all i . Now we can also exclude $i = 6$. The solution is then to select the larger of π_7 and π_5 .

If the profit function is submodular, then we start by comparing profits of the two configurations with the fewest facilities (1 and 2). If $\pi_2 < \pi_1$ then position $y_3^i = 0$ for all i . We can eliminate $i = 3, 4, 8$ (blue dashed rectangle) from consideration. Next we would evaluate 5 and 6: Whichever is higher will dominate 7 so we do not need to evaluate 7 either. Finally, we select the larger of π_1 and the winner of 5 vs 6.

In both cases we only need to evaluate four options, half of the number required by brute force. The examples above showed case where squeezing works, but there could be scenarios where it does not work effectively. For example, in the submodular case, suppose $\pi_2 > \pi_1$, $\pi_6 > \pi_1$ and $\pi_5 > \pi_1$. Then we can not rule out any locations in the initial squeezing.

B Proof that UFLP is the limiting case of AES

Proof. For Box 4.1 since x is binary as in eq (27), let the selected set (if nonempty)¹⁴ be

$$S_n \equiv \{\ell \in L : x_{\ell n} = 1\} \neq \emptyset, \quad k_n \equiv |S_n| = \sum_{\ell \in L} x_{\ell n} \geq 1.$$

We have

$$\sum_{\ell \in L} c_{\ell n}^{-\theta} x_{\ell n} = \sum_{\ell \in S_n} c_{\ell n}^{-\theta}.$$

Define the minimum cost among the selected paths,

$$\underline{c}_n \equiv \min_{\ell \in S_n} c_{\ell n}.$$

Then we have the following inequalities, given $-(1 - \eta)/\theta > 0$,

$$\begin{aligned} \underline{c}_n^{-\theta} &\leq \sum_{\ell \in S_n} c_{\ell n}^{-\theta} && \leq k_n \underline{c}_n^{-\theta} \\ [\underline{c}_n^{-\theta}]^{-(1-\eta)/\theta} &\leq \left[\sum_{\ell \in L} c_{\ell n}^{-\theta} x_{\ell n} \right]^{-(1-\eta)/\theta} && \leq [k_n \underline{c}_n^{-\theta}]^{-(1-\eta)/\theta} \\ \underline{c}_n^{1-\eta} &\leq \left[\sum_{\ell \in L} c_{\ell n}^{-\theta} x_{\ell n} \right]^{-(1-\eta)/\theta} && \leq k_n^{-(1-\eta)/\theta} \underline{c}_n^{1-\eta}. \end{aligned}$$

When $\theta \rightarrow \infty$, $k_n^{-(1-\eta)/\theta} \rightarrow 1$. By squeezing,

$$\left[\sum_{\ell \in L} c_{\ell n}^{-\theta} x_{\ell n} \right]^{-(1-\eta)/\theta} \xrightarrow{\theta \rightarrow \infty} \underline{c}_n^{1-\eta}.$$

¹⁴When the selected set is empty, $S_n = \emptyset$, the problem is trivial because in both formulations, total profit is zero.

In Box 2.2 because of eq (7) and the implied single-sourcing, we have

$$\sum_{\ell \in L} c_{\ell n}^{1-\eta} x_{\ell n} = \underline{c}_n^{1-\eta}.$$

Therefore, with different specifications and constraints imposed on $x_{\ell n}$, Box 4.2 is the limiting case of Box 2.2 when $\theta \rightarrow \infty$.¹⁵

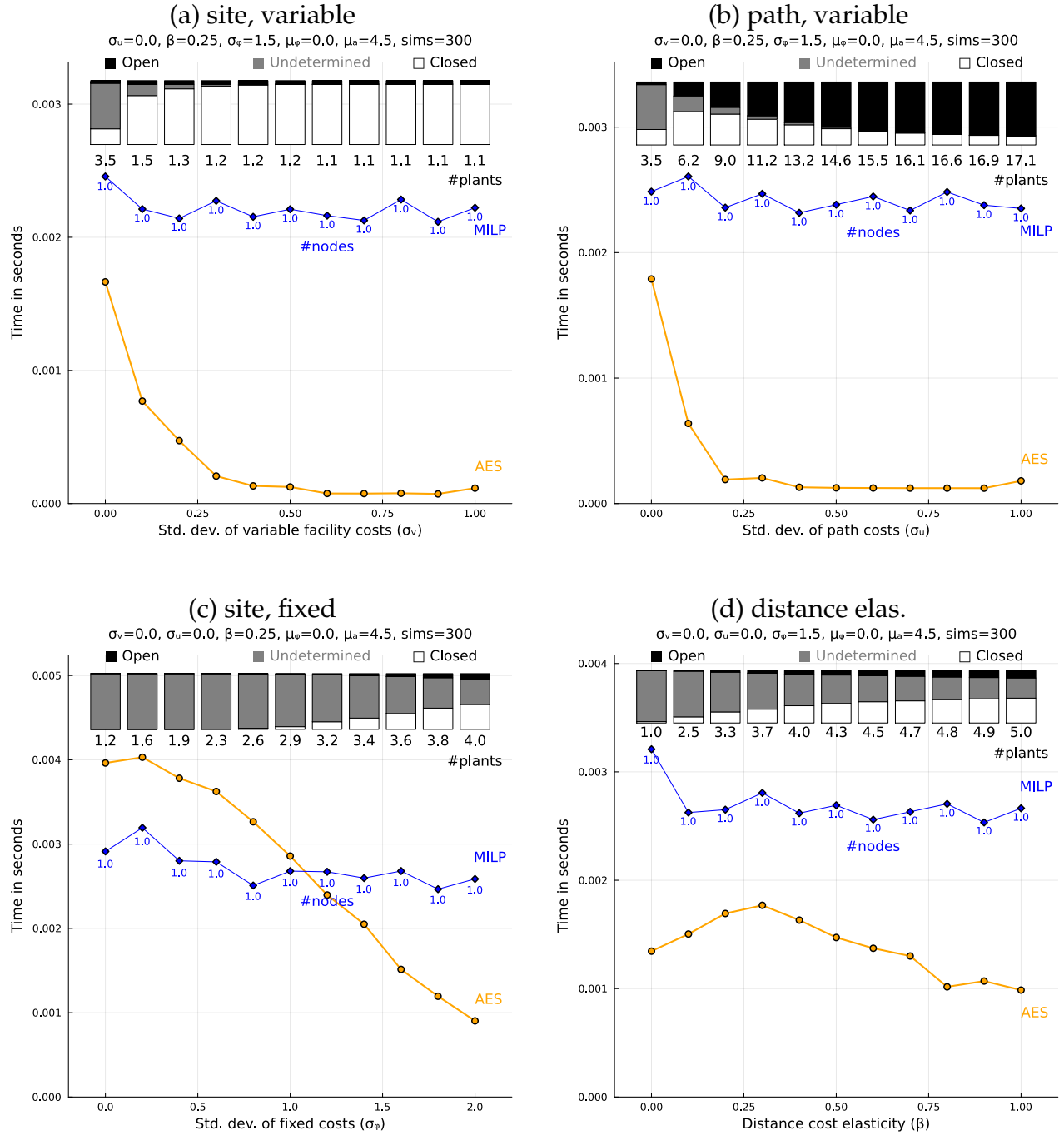
□

C Other figures

Figure C2 and C3 are comparable to Figure 5, but with $L = 20$ and $L = 80$. Critical values shown in these plots correspond to Table 2.

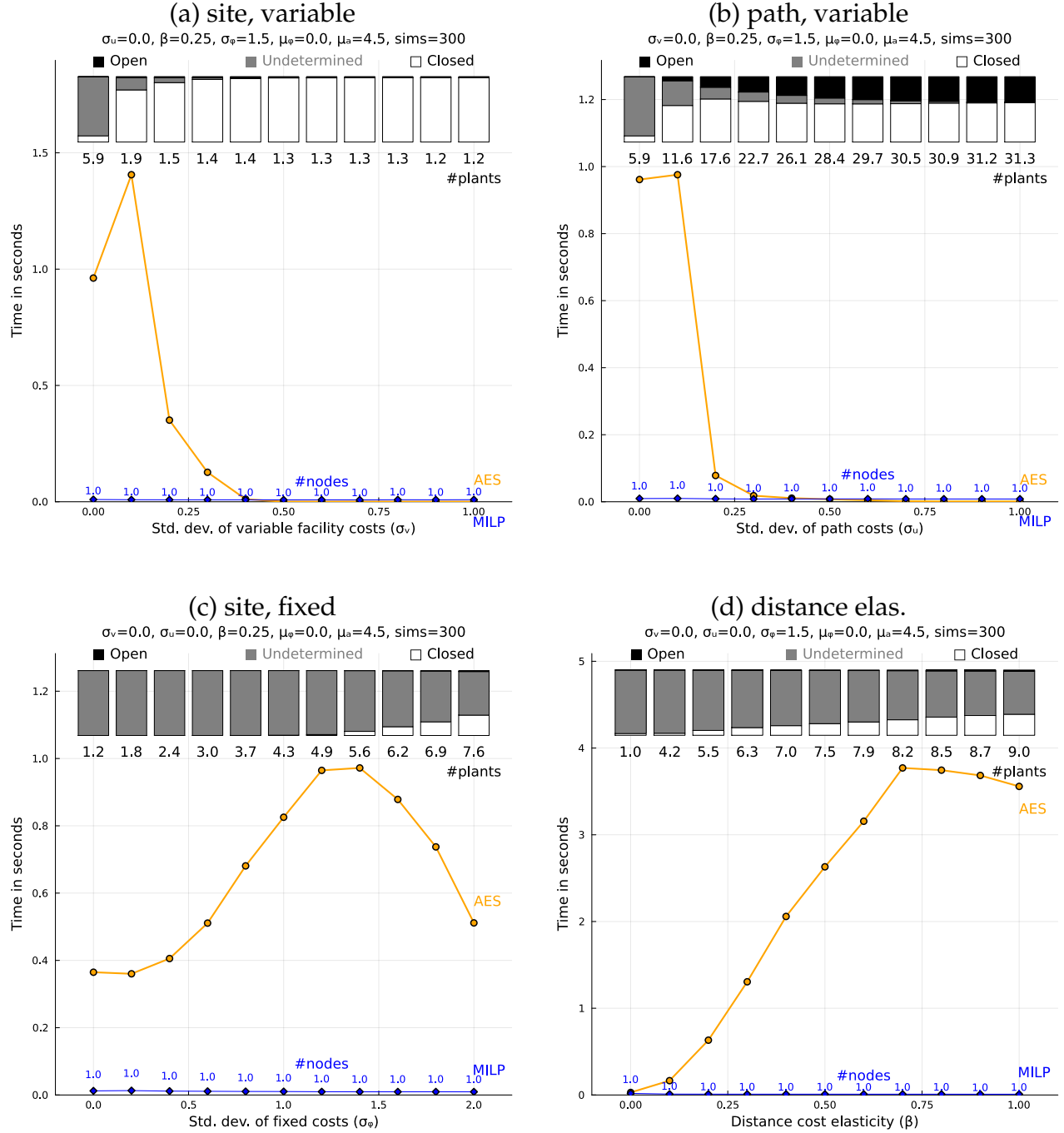
¹⁵An alternative way to prove is to take log and then apply L'Hôpital's rule, which I find it not as straightforward as the squeezing approach.

Figure C2: How heterogeneity affects relative performance of MILP and squeezing



Notes: $L = 20, N = 40$. We vary each parameter while keeping the rest fixed.

Figure C3: How heterogeneity affects relative performance of MILP and squeezing



Notes: $L = 80, N = 40$. We vary each parameter while keeping the rest fixed.